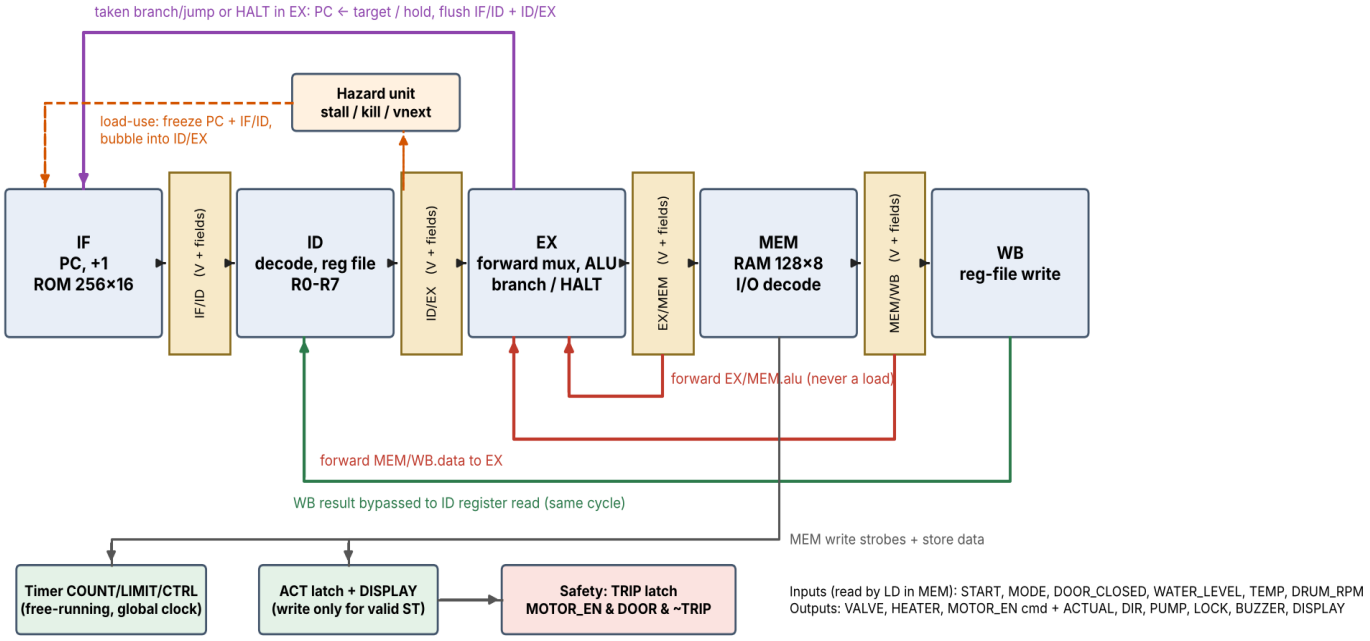
Architecture: five-stage pipelined washing-machine controller

Single-issue, in-order IF - ID - EX - MEM - WB with IF/ID, ID/EX, EX/MEM and MEM/WB registers, each carrying a valid bit V. One global rising-edge clock; asynchronous active-high reset. The circuit is built hierarchically from 24 named Logisim subcircuits (top: **washing_pipeline**); every net is labelled and every architectural write is qualified by the stage's V bit.



Top-level subcircuits

Instance	Subcircuit	Role
IF	fetch	PC register, +1 incrementer (gate-level adder), next-PC mux, instruction ROM, sticky HALTED flag
IF_ID / ID_EX / EX_MEM / MEM_WB	ifid, idex, exmem, memwb	Pipeline registers (built-in Register parts, enable/clear driven by stall/flush/reset)
ID	decode	Opcode decode (one AND per opcode), control word, usesA/usesB source-use bits, field extraction
RF	regfile	R0-R7 (no hard-wired zero), 3-to-8 write decoder, two 8:1 read mux trees, WB-to-ID same-cycle bypass
HZ	hazard	Load-use stall, flush priority (kill), ID/EX valid-next
EX	execute	Two forwarding units (fwd), operand mux, ALU (add/sub/and/pass/SLTU), zero detect, branch / jump / HALT resolve
MEM	mem_stage	Address decode, RAM (128x8) + tri-state write driver, peripheral read mux, write strobes
IO / TMR / SAFE	periph, timer, safety	ACT + DISPLAY latches, timer, TRIP latch and combinational motor gate

Pipeline-register fields and control word

Register	Fields (width)
PC / HALTED	PC (8), halted (1). Reset: PC = 0, halted = 0.
IF/ID	V (1), IR (16)
ID/EX	V; Aval (8), Bval (8), imm (8); fA (3, doubles as destination), fB (3); aluOp (3); regWrite, memRead, memWrite, aluSrcImm, brZ, brNZ, jmp, halt, usesA, usesB
EX/MEM	V; regWrite, memRead, memWrite; dest (3); alu (8: result or memory address); stData (8: forwarded A = store data)
MEM/WB	V; regWrite; dest (3); data (8: load data or ALU result, chosen by memRead in MEM)

Instr	regW	memR	memW	srcImm	aluOp	brZ	brNZ	jmp	halt	usesA	usesB
LDI	1	0	0	1	0	0	0	0	0	0	0
MOV	1	0	0	0	0	0	0	0	0	0	1
ADD	1	0	0	0	1	0	0	0	0	1	1
SUB	1	0	0	0	2	0	0	0	0	1	1
AND	1	0	0	0	3	0	0	0	0	1	1
SLTU	1	0	0	0	4	0	0	0	0	1	1
ADDI	1	0	0	1	1	0	0	0	0	1	0
LD	1	1	0	0	0	0	0	0	0	0	1
ST	0	0	1	0	0	0	0	0	0	1	1
BEQZ	0	0	0	0	0	1	0	0	0	1	0
BNEZ	0	0	0	0	0	0	1	0	0	1	0
JMP	0	0	0	0	0	0	0	1	0	0	0
HALT	0	0	0	0	0	0	0	0	1	0	0
NOP/0xE/0xF	0	0	0	0	0	0	0	0	0	0	0

aluOp: 0 PASSB, 1 ADD, 2 SUB, 3 AND, 4 SLTU. Store data = forwarded A, store address = forwarded B (both used in EX). Branch test uses forwarded A.

Forwarding, stall and flush behaviour

• **Forwarding (per operand X in {A,B}, only if usesX):** if EX/MEM.V & regWrite & *not memRead* & dest = fX use EX/MEM.alu (youngest producer); else if MEM/WB.V & regWrite & dest = fX use MEM/WB.data; else the ID/EX value. A load in EX/MEM is never forwarded; the stall rule guarantees its consumer is not in EX. The register file bypasses WB data to a same-cycle ID read, so a WB write is visible to ID.

• **Load-use:** loadUse = ID/EX.V & ID/EX.memRead & IF/ID.V & ((usesA & fA = ID/EX.dest) | (usesB & fB = ID/EX.dest)) using the *decoded* usesA/usesB of the instruction in ID, so fields an instruction does not read never stall (e.g. LDI, JMP, NOP, the B field of ADDI/BEQZ). On stall: PC and IF/ID hold, ID/EX receives V = 0 (bubble), older stages advance. Exactly one bubble.

• **Flush priority:** redirect = ID/EX.V & (jmp | brZ & A=0 | brNZ & A≠0); haltEX = ID/EX.V & halt; kill = redirect | haltEX; stall = loadUse & ~kill. On kill the next IF/ID.V and ID/EX.V are 0 and PC <- target (branch) or holds (HALT): a taken branch costs exactly two wrong-path instructions.

• **Next-state:** PC: redirect ? target : (stall | halted | haltEX) ? hold : PC+1. IF/ID: stall ? hold : V = ~kill & ~halted. ID/EX: V = IF/ID.V & ~kill & ~stall. EX/MEM and MEM/WB always advance.

• **Store commits once:** EX/MEM is reloaded every cycle and MEM never stalls, so a store is in MEM for exactly one cycle; RAM / ACT / DISPLAY / timer enables are EX/MEM.V & memWrite & region select, so bubbles and flushed instructions (V = 0) cannot write. Register writes are MEM/WB.V & regWrite.

Reset and HALT

Reset (active high, asynchronous clear of the state registers): PC = 0, HALTED = 0, all four pipeline valid bits = 0, ACT = 0 (all actuator requests off), DISPLAY = 0, timer EN/COUNT/LIMIT/MATCH = 0, TRIP = 0, START_PREV = 0. R0-R7 and RAM are not cleared (programs initialise them). **HALT:** a valid HALT recognised in EX sets the sticky HALTED flag, flushes IF/ID and ID/EX and stops PC; older instructions in MEM/WB complete normally; the controller stays halted until reset. A wrong-path HALT is killed by the redirect before it reaches EX, so haltEX is never raised for it. The timer and the safety gate keep running while halted.

Safety interlock

```
ActualMotorEnable = CPU_MOTOR_EN & DOOR_CLOSED & ~TRIP          (purely combinational AND gate, no clock)
startRise = START & ~START_PREV                                  START_PREV <- START   (flop, rising edge)
set       = CPU_MOTOR_EN & ~DOOR_CLOSED                          (door sampled open while the request is active)
rearm     = ~CPU_MOTOR_EN & startRise & DOOR_CLOSED              (software cleared the request AND new START edge, door closed)
TRIP      <- set | (TRIP & ~rearm)                               (one flop, same global rising edge; set wins over rearm)
```

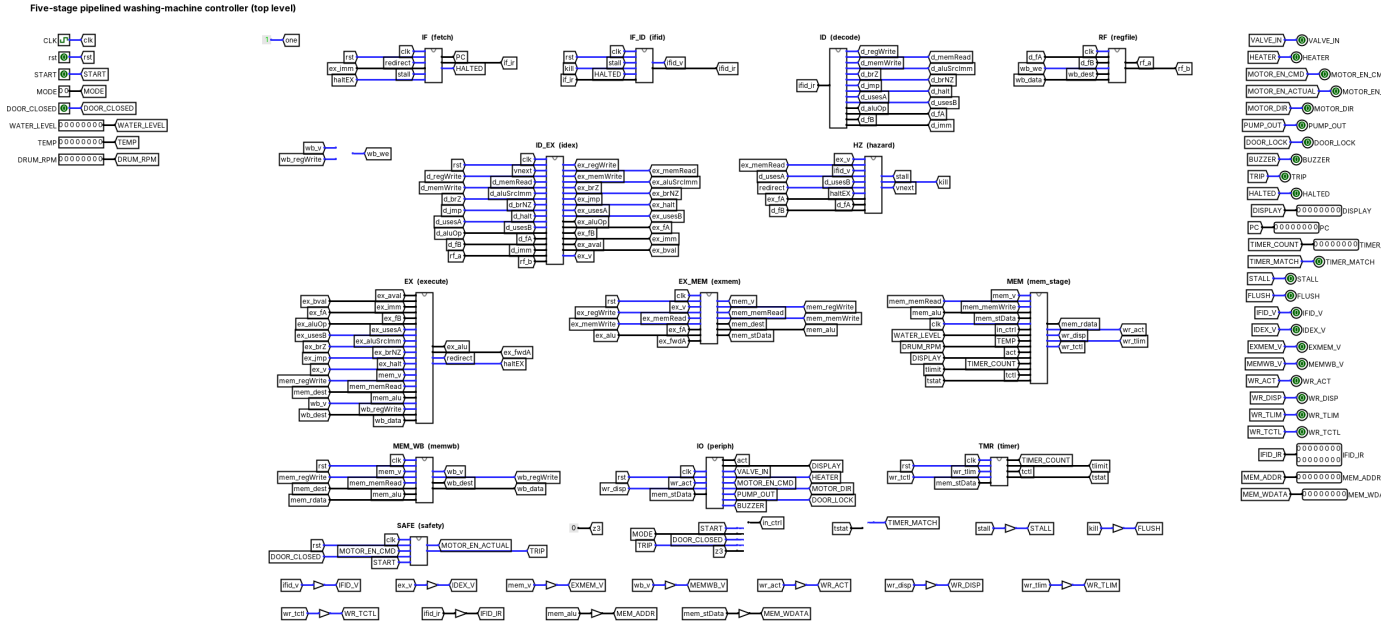
Clocking and independence: TRIP and START_PREV are ordinary rising-edge flops on the global clock (asynchronously cleared by reset). The motor gate is a combinational AND of the *output latch* bit CPU_MOTOR_EN with the live DOOR_CLOSED pin and NOT TRIP, so opening the door drops ActualMotorEnable immediately, before any clock edge and without CPU polling; TRIP then latches at the next edge and survives reclosing. CPU_MOTOR_EN comes only from the valid-qualified ACT register write, so a flushed instruction cannot request the motor. A cleared door sensor alone never restarts the motor: it needs software to drop the request and a fresh START low-to-high edge with the door closed. Both CPU_MOTOR_EN and MOTOR_EN_ACTUAL are separate output pins.

Timer

8-bit COUNT register with a gate-level incrementer (ripple full-adder chain, no Counter part), LIMIT register, EN flag and sticky MATCH; clear has priority; clocked only by the global clock, so it keeps counting through stalls and after HALT (see memory_map.pdf).

Design-rule compliance

Logisim components used in the whole design: AND Gate x90, Buffer x20, Clock x1, Constant x19, Controlled Buffer x1, NOR Gate x5, NOT Gate x31, OR Gate x24, Pin x281, RAM x1, ROM x1, Register x48, Splitter x47, XOR Gate x5. 24 circuits. No built-in Adder, Subtractor, Multiplexer, Decoder, Comparator, Shifter or Counter is used; the ALU, muxes, decoders, register-file selection, incrementers and comparators are made from gates and the project's own tested subcircuits. The Controlled Buffer appears only as the write driver of the bidirectional RAM data port.



Top-level circuit as drawn in Logisim (rendered from washing_pipeline.circ; overview only, open the .circ for detail).